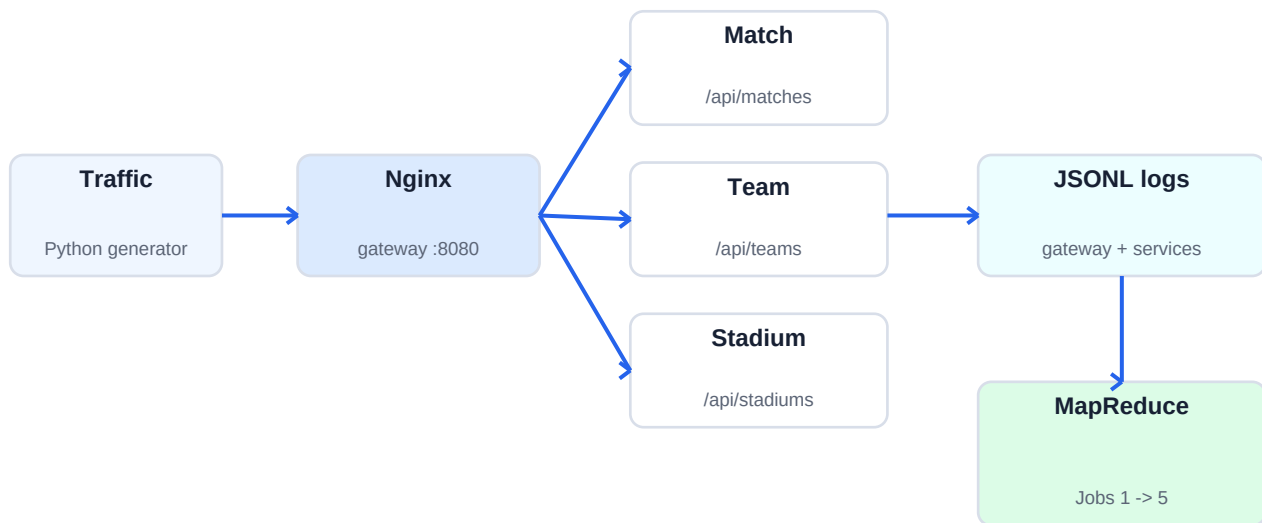


World Cup Log Analytics

Nginx API Gateway and Hadoop Streaming MapReduce
Cloud Computing Final Project - Phase 1



Implementation: mandatory Phase 1 pipeline

Team identification: not supplied in this workspace

Report status: implementation documented; live evidence pending Docker run

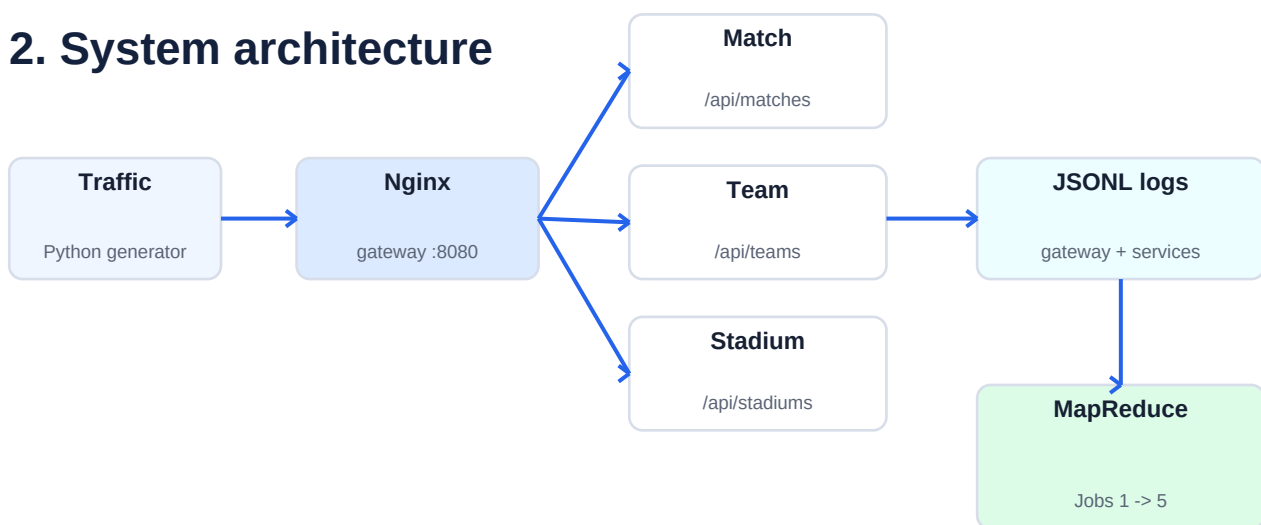
1. Executive summary

The system provides three containerized FastAPI services for World Cup match, team, and stadium data. Nginx is the only public entry point. It routes requests, forwards correlation headers, and writes a structured gateway log. Each backend writes a correlated, entity-aware service log. Five Hadoop Streaming jobs transform the logs into required CSV statistics and a final JSON summary.

Scope and current verification

- All mandatory source code, configuration, and automation are implemented.
- The deterministic end-to-end fixture validates calculations and tie-breaking.
- The local suite currently contains 55 passing tests.
- The genuine 100,000-request data run remains pending on a Docker-capable machine.

2. System architecture



The root Compose project creates one private bridge network. Nginx publishes host port 8080; backends expose port 8000 only inside the network. Persistent host mounts retain gateway and service logs. A separate Hadoop Compose project mounts the repository at /project inside the NameNode so the pipeline can access source logs and write host-visible outputs.

Component	Responsibility	Persistent artifact
Nginx	Route, default and forward headers, gateway timing	data/nginx/nginx_access.log
Backends	Serve data and log entity-level outcomes	data/service_logs/*.log
Traffic generator	Create varied, correlated load through Nginx	No trace file
Hadoop	Execute five batch analytics stages	outputs/job1..job4
Final report job	Combine analytics and prediction metadata	outputs/final/summary.json

3. API gateway and services

Route	Backend	Entity input	Expected outcomes
/api/matches	match-service	date	200, 400, 404, forced 500
/api/teams	team-service	name	200, 404, forced 500
/api/stadiums	stadium-service	name or city	200, 400, 404, forced 500

Header propagation

Nginx resolves effective request ID, country, and scenario values. Missing request IDs use Nginx's generated request identifier; country defaults to Unknown and scenario defaults to normal. The same effective values are logged at the gateway and forwarded to the backend, enabling request-level correlation.

```
curl -H "X-Request-ID: manual-001" \ -H "X-Client-Country: Iran" \ -H "X-Scenario: normal" \
"http://localhost:8080/api/teams?name=Argentina"
```

4. Structured logging

Gateway schema

```
timestamp, request_id, client_ip, client_country, scenario, method, path, service, status_code,
request_time_sec, user_agent
```

Backend schema

```
timestamp, request_id, client_country, scenario, service, endpoint, entity_type, entity_value,
status_code, processing_time_ms, event_type
```

Both schemas use one complete JSON object per line. Gateway time is a numeric string in seconds; service processing time is numeric milliseconds. The validator checks syntax, types, required fields, duplicate IDs, gateway/backend correlation, status agreement, service coverage, countries, scenarios, entity diversity, and timing diversity.

5. Traffic generation

The standard-library generator uses bounded concurrency, unique request IDs, deterministic seeds, timeouts, and an in-memory summary. A coverage prefix guarantees required services and scenarios before weighted random traffic creates intentionally uneven popularity. All requests target Nginx; no backend URL is accepted.

```
python traffic-generator/generate.py --requests 100000 \ --nginx-url http://localhost:8080
```

6. Hadoop Streaming design

Job	Input	Transformation	Output
1	Raw gateway + service JSONL	Validate, normalize, tag invalid rows	3 CSV files
2	Clean gateway rows	Service, endpoint, scenario metrics	3 CSV files
3	Clean service rows	Country/entity request counts	3 CSV files
4	Job 3 counts	Deterministic popular entity selection	3 CSV files
5	Jobs 2-4 + predictions	Cross-domain final aggregation	summary.json

Determinism and reruns

Each job uses one reducer. Composite keys are compact JSON arrays, and ties select the case-insensitive lexical minimum followed by original text order. The pipeline uses an isolated /phase1 HDFS namespace, deletes only known prior input and job-output paths, and materializes headers exactly once. A repeated local run produces byte-identical artifacts.

7. Testing and reproducibility

Test area	Coverage
Containers and Compose	Images, health checks, mounts, private network, host ports
Nginx	Routes, upstreams, headers, JSONL fields, health and 404 responses
Service logging	Success, validation, not found, slow, forced errors, concurrent writes
Traffic	Gateway-only URLs, reproducibility, weighting, scenarios, concurrency
Log validation	Malformed rows, types, correlation, duplicates, diversity
MapReduce	Five-stage flow, malformed/empty input, metrics, ties, output schemas
Final run	Confirmation guard, preserved artifacts, cross-file summary consistency

```
python -m unittest discover -s tests -v python scripts/verify_local_e2e.py bash
scripts/run_final_data.sh --confirm-final-run
```

Clean-run safety

The application is stopped before log truncation. Only four known log paths are truncated in place, preventing deletion of an actively open Nginx file. The final workflow requires explicit confirmation and a request count of at least 100,000. The independent artifact verifier recomputes every final summary value from intermediate CSV files.

8. Live-run evidence

The assignment requires evidence from the genuine final Docker and Hadoop run. Available images are embedded below. A missing image is reported explicitly and must be captured before submission.

Healthy application containers

Pending live evidence: 01-containers.png

Successful request through Nginx

Pending live evidence: 02-nginx-request.png

Correlated Nginx gateway record

Pending live evidence: 03-nginx-log.png

Correlated backend service record

Pending live evidence: 04-service-log.png

100,000-request traffic summary

Pending live evidence: 05-traffic-generator.png

Five successful Hadoop Streaming jobs

Pending live evidence: 06-hadoop-streaming.png

Representative intermediate CSV output

Pending live evidence: 07-intermediate-output.png

Final summary.json

Pending live evidence: 08-final-summary.png

9. Results interpretation

The final summary identifies traffic concentration, reliability, latency, and entity popularity. `total_requests` must equal cleaned gateway rows, cleaned service rows, and the sum of service totals. The highest error-rate service uses the Job 2 error ratio. The slowest endpoint uses average gateway response time. Overall entity fields sum Job 3 counts across countries, while `popular_team_by_country` comes from Job 4.

Result availability

No final submission metrics are asserted in this draft because the Docker-based final run has not occurred in the current environment. After that run, rebuild this report so the live evidence and actual summary are included. This avoids presenting fixture data as real experimental evidence.

10. Limitations and future work

- The mandatory implementation is batch-oriented; Spark Structured Streaming is optional and not implemented.
- The supplied Hadoop image installs Python at startup and therefore needs package-repository access on a first clean run.
- The report requires live evidence and team identification before submission.

11. Conclusion

The implementation establishes a reproducible path from API traffic to correlated structured logs, Hadoop Streaming analytics, and a cross-validated final report. Static contracts, unit tests, a hand-calculated end-to-end fixture, guarded execution, and final artifact checks reduce the risk of schema drift or inconsistent submission data. The remaining operational action is to run the workflow on Docker, capture evidence, and rebuild the PDF.